تخصيص RouterOS — HotSpot — وثائق MikroTik

• ترجمة الشرح فقط إلى العربية. المصطلحات التقنية، أسماء الملفات، المتغيرات، الأوامر، وأمثلة الكود تبقى كما في الأصل.

ملاحظة الموقع

هذا الموقع التوثيقي مجمد، ولن تجرى عليه تعديلات إضافية.

تخصيص (HotSpot (Hotspot customisation

• أنشأه Normunds R.، آخر تحديث بواسطة GG في 08 Mar 2024 — قراءة تقارب 19 دقيقة

المتطلبات (Requirements)

• HotSpot captive portal

مقدمة (Introduction)

على hotspot router، ثم عين قيمة الخاصية `"html-override-directory"` كمسار إلى مجلد HTML المخصص الجديد. قد تفسد الصفحات بحذف المتغيرات أو أجزاء حيوية أخرى. بعد الانتهاء من تعديل المحتوى، ارفع المحتوى المعدل إلى مجلد مخصص وتعديل الصفحات كما تشاء مستفيدا من معلومات هذا القسم من الدليل. يفضل تحرير الملفات يدويا؛ لأن أدوات تحرير HTML الآلية مباشرة بعد إنشاء server profile. يمكن الوصول إلى هذا المجلد بالاتصال بالموجه عبر FTP client. يمكنك نسخ هذا المجلد في HotSpot server profile ضمن `ip hotspot profile/`. تنسخ صفحات servlet الافتراضية إلى المجلد `"hotspot"` تماما من صفحات servlet لكل HotSpot server لديك، وذلك بتحديد المجلد عبر الخاصية `"html-override-directory"` يمكنك إنشاء مجموعة مختلفة

ملاحظة: إذا كان مسار `"html-override-directory"` مفقودا أو فارغا، فسيعود hotspot server إلى ملفات HTML الافتراضية.

الصفحات المتاحة (Available Pages)

صفحات HTML servlet الرئيسية التي تعرض للمستخدم:

• `redirect.html` — تعيد توجيه المستخدم إلى URL صفحة آخر (مثلا إلى login)

• `login.html` — صفحة تسجيل الدخول التي تطلب username و password. قد تقبل المعاملات التالية:

• `username` — اسم المستخدم

• `password` — إما كلمة مرور plain-text (في حالة PAP authentication) أو MD5 hash لـ متغير `chap-id` المرور و e-mail address لمستخدمي trial تستخدم هذه القيمة كـ CHAP challenge (في حالة CHAP authentication). وكلمة

• `dst` — الـ URL الأصلي المطلوب قبل الـ redirect. يفتح بعد نجاح login

• `popup` — هل تفتح نافذة status عند نجاح login

• `radius <id>` — إرسال الـ attribute المحدد كـ text string إلى RADIUS server (عند استخدام RADIUS authentication؛ وإلا يهمل)

• `radius <id> u` — إرسال الـ attribute المحدد كـ unsigned integer إلى RADIUS server (عند استخدام RADIUS authentication؛ وإلا يهمل)

• `radius <id>-<vendor> ` — إرسال الـ attribute المحدد مع vendor ID كـ text string إلى RADIUS server (استخدام RADIUS authentication عند؛ وإلا يهمل)

• `radius <id>-<vendor> u` — إرسال الـ attribute المحدد مع vendor ID كـ unsigned integer إلى RADIUS server (عند استخدام RADIUS authentication؛ وإلا يهمل)

• `md5.js` — JavaScript لتجزئة كلمة المرور بـ MD5. يستخدم مع طريقة login من نوع `http-chap`

• `alogin.html` — الصفحة المعروضة بعد أن يسجل client دخوله. تفتح صفحة status المطلوبة أصلا (قبل إعادة التوجيه إلى HotSpot login page) وتعيد توجيه المتصفح إلى الصفحة

• `status.html` — تعرض إحصاءات الـ client. صفحة الحالة؛ ويمكنها أيضا عرض advertisements تلقائيا

• `logout.html` — تعرض بعد logout. صفحة تسجيل الخروج؛ النهائية للجلسة المنتهية. قد تقبل المعاملات الإضافية التالية: تعرض الإحصاءات

• `erase-cookie` — هل تمسح cookies من HotSpot server عند logout (يمنع cookie تلقائيا عبر login) مفيد في بيئات multiuser) في المرة التالية من نفس المتصفح؛

• `error.html` — تعرض عند الأخطاء القاتلة فقط. صفحة الأخطاء؛

وهناك صفحات أخرى عند الحاجة لمزيد من التحكم:

• `rlogin.html` — تعيد توجيه الـ client من URL آخر إلى login page إذا كانت authorization مطلوبة للوصول إلى ذلك الـ URL

• `rstatus.html` — مشابهة لـ `rlogin.html`، لكن عندما يكون الـ client والـ URL مسجل الدخول أصلا والأصلي غير معروف

• `radvert.html` — تعيد توجيه الـ client إلى رابط advertisement المجدول

• `flogin.html` — تعرض بدلا من `login.html` عند حدوث خطأ (مثل username أو password غير صالح)

• `fstatus.html` — تعرض بدلا من status page عند طلب redirect بينما الـ client غير مسجل الدخول

• `flogout.html` — تعرض بدلا من logout page عند طلب redirect بينما الـ client غير مسجل الدخول

خدمة صفحات Servlet (Serving Servlet Pages)

يتعرف HotSpot servlet على 5 أنواع طلبات مختلفة:

1. طلب لمضيف بعيد (remote host)

• إذا كان المستخدم مسجل الدخول

وحان موعد عرض advertisement، تعرض `radvert.html`. هذه الصفحة تعيد التوجيه إلى صفحة advertisement المجدولة

• إذا كان المستخدم مسجل الدخول ولا يوجد advertisement مجدول لهذا المستخدم، تخدم الصفحة المطلوبة

• إذا لم يكن المستخدم مسجل الدخول، لكن الوجهة مسموحة عبر walled garden، فيخدم الطلب أيضا

• إذا لم يكن المستخدم مسجل الدخول، والوجهة ممنوعة عبر walled garden، تعرض `rlogin.html`؛ وإن لم توجد، تستخدم `redirect.html` لإعادة التوجيه إلى login page

2. طلب لـ `"/"` على مضيف HotSpot

• إذا كان المستخدم مسجل الدخول، تعرض `rstatus.html`؛ وإن لم توجد، تستخدم `redirect.html` لإعادة التوجيه إلى status page

• إذا لم يكن المستخدم مسجل الدخول، تعرض `rlogin.html`؛ وإن لم توجد، تستخدم `redirect.html` لإعادة التوجيه إلى login page

3. طلب لصفحة `"login/"`

• إذا نجح login (أو كان المستخدم مسجلا مسبقا)، تعرض `alogin.html`؛ وإن لم توجد، تستخدم `redirect.html` لإعادة التوجيه إلى الصفحة المطلوبة أصلا أو إلى status page (إن لم تعط الوجهة الأصلية)

• إذا لم يكن المستخدم مسجل الدخول (لم يرسل username ولم تظهر رسالة خطأ)، تعرض `login.html`

• إذا فشلت عملية login (وجدت رسالة خطأ)، تعرض `flogin.html`؛ وإن لم توجد، تستخدم `login.html`

• في حالة الأخطاء القاتلة، تعرض `error.html`

4. طلب لصفحة `"status/"`

• إذا كان المستخدم مسجل الدخول، تعرض `status.html`

• إذا لم يكن مسجل الدخول، تعرض `fstatus.html`؛ وإن لم توجد، تستخدم `redirect.html` لإعادة التوجيه إلى login page

5. طلب لصفحة `'logout/'`

• إذا كان المستخدم مسجل الدخول، تعرض `logout.html`

• إذا لم يكن مسجل الدخول، تعرض `flogout.html`؛ وإن لم توجد، تستخدم `redirect.html` لإعادة التوجيه إلى login page

ملاحظة: إذا تعذر تلبية الطلب باستخدام الصفحات المخزنة على FTP server الخاص بالموجه، تعرض Error 404

هناك طرق عديدة لتخصيص مظهر صفحات HotSpot authentication:

قابلة للتعديل بسهولة. تخزن على FTP server الخاص بالموجه في المجلد الذي تختاره لـ HotSpot server profile المعني.
• الصفحات

License Agreement؛ يمكن استخدام قيم جاهزة عامة لجميع المستخدمين أو MAC address كـ username و password) إلى واحد (username أو password؛ مثلا يمكن استخدام MAC address للـ client كقيمة أخرى) أو حتى إلى صفر (مثل
• بتغيير المتغيرات التي يرسلها الـ client إلى HotSpot servlet، يمكن تقليل عدد الحقول المطلوبة

client إليه حتى لا يدخل يدويا. بعد التسجيل، يجب أن يعدل الخادم قاعدة بيانات RADIUS لتمكين الـ client من login لمدة معينة.
• قد تتم registration على خادم مختلف (مثلا خادم يقبل Credit Cards). يمكن تمرير MAC address للـ

و `chap-id` و `chap-challenge` و `popup` المتاحة في login page فقط). مثال لإظهار رابط إلى login page:
مخزن على HotSpot server (باستثناء traffic counters المتاحة في status page فقط، ومتغيرات `error` و `error-orig` يوصل إليه كـ `'/'` أو `'login/'` أو `'status/'` أو `'logout/'`، وكذلك أي ملف نصي أو HTML (`.txt` أو `.htm` أو `.html`) حيث `"var_name"` هو اسم المتغير (بدون علامات اقتباس). يمكن استخدام هذا التركيب في أي ملف HotSpot HTML لإدراج متغير في مكان ما داخل ملف HTML، تستخدم الصيغة `$(var_name)`

```
<a href="$(link-login)">login</a>
```

المتغيرات (Variables)

في كل صفحات servlet، لكن بعضها قد يكون فارغا وقت الوصول إليه (مثلا لا يوجد uptime قبل أن يسجل المستخدم دخوله). HotSpot تلقائيا بالقيم المناسبة. لمعظم المتغيرات يوجد مثال لقيمة محتملة بين أقواس. جميع المتغيرات الموصوفة صالحة متغيراتا لعرض قيم خاصة بالمستخدم. تظهر أسماء المتغيرات فقط في مصدر HTML لصفحات servlet — ويستبدلها Servlet تستخدم جميع صفحات Servlet HTML

قائمة المتغيرات المتاحة (List of available variables)

https يمكنك بناء الرابط بطريقة أخرى، مثلا لـ `link-status$` استخدم: `https://$(hostname)/$(target-dir)status`
ملاحظة: بعض المتغيرات تستخدم http URL مضمنا بشكل ثابت؛ إذا كنت تستخدم

متغيرات الخادم العامة (Common server variables):

• `hostname` — DNS name أو IP address (إن لم يعط HotSpot Servlet لـ (DNS name `("hotspot.example.net"`)

• `identity` — اسم الهوية في RouterOS (`"MikroTik"`)

• `login-by` — طريقة authentication التي استخدمها المستخدم

• `plain-passwd` — تمثيل `"yes/no"` لما إذا كانت طريقة HTTP-PAP login مسموحة (`"no"`)

• `server-address` — عنوان HotSpot server (`"10.5.50.1:80"`)

• `ssl-login` — تمثيل `"yes/no"` لما إذا استخدم HTTPS للوصول إلى صفحة servlet هذه (`"no"`)

• `server-name` — اسم HotSpot server (يضبط في قائمة `/ip hotspot` عبر الخاصية `name`)

الروابط (Links):

• `link-login` — رابط إلى login page الأصلي المطلوب URL يتضمن الـ (`"http://10.5.50.1/login?dst=http://www.example.com/"`)

• `link-login-only` — رابط إلى login page دون الـ URL الأصلي (`"http://10.5.50.1/login"`)

• `link-logout` — رابط إلى logout page (`"http://10.5.50.1/logout"`)

• `link-status` — رابط إلى status page (`"http://10.5.50.1/status"`)

• `link-orig` — الـ URL الأصلي المطلوب (`"http://www.example.com/"`)

معلومات عامة عن الـ client:

• `domain` — domain name للمستخدم (`"example.com"`)

• `interface-name` — اسم واجهة HotSpot الفعلية (في حالة bridged interfaces يرجع اسم bridge port الفعلي)

• `ip` — عنوان IP للـ client (`"10.5.50.2"`)

• `logged-in` — `"yes"` إذا كان المستخدم مسجل الدخول، وإلا `"no"` (`"yes"`)

• `mac` — عنوان MAC 01:23:45:67:89:AB"`)للمستخدم (`"

• `trial` — تمثيل `"yes/no"` إذا كان للمستخدم وصول إلى trial time. لما انتهت مدة trial يصبح القيمة `"no"`

• `username` — اسم المستخدم (`"John"`)

• `host-ip` — عنوان IP للـ client من جدول `/ip hotspot host`

• `vlan-id` — يمثل ID لواجهة VLAN التي يتصل منها الـ client

معلومات حالة المستخدم (User status information):

• `idle-timeout` — 20"`)مهلة الخمول (`" (أو `""` إن لم توجد `"m

• `idle-timeout-secs` — مهلة الخمول بالثواني (`"88"` أو `"0"` إن لم توجد)

• `limit-bytes-in` — حد البايتات للإرسال (`"1000000"` أو `"---"` إن لم يوجد حد)

• `limit-bytes-out` — حد البايتات للاستقبال (`"1000000"` أو `"---"` إن لم يوجد حد)

• `refresh-timeout` — مهلة تحديث status page (`"1m30s"` أو `""` إن لم توجد)

• `refresh-timeout-secs` — مهلة تحديث status page بالثواني (`"90"s" أو `"0"` إن لم توجد)

• `session-timeout` — 5"`)الوقت المتبقي للجلسة (`"h (أو `""` إن لم يوجد `"

• `session-timeout-secs` — الوقت المتبقي للجلسة بالثواني (`"3475"` أو `"0"` إن لم يوجد)

• `session-time-left` — 5"`)الوقت المتبقي للجلسة (`"h (أو `""` إن لم يوجد `"

• `session-time-left-secs` — الوقت المتبقي للجلسة بالثواني (`"3475"` أو `"0"` إن لم يوجد)

• `uptime` — 10"`)مدة الجلسة الحالية (`"h2m33s"

• `uptime-secs` — 125"`)مدة الجلسة الحالية بالثواني (`"

عدادات الحركة (Traffic counters) — متاحة في status page فقط:

• `bytes-in` — 15423"`)عدد البايتات المستلمة من المستخدم (`"

• `bytes-in-nice` — 15423"`)عرض مقروء لعدد البايتات المستلمة من المستخدم (`"

• `bytes-out` — 11352"`)عدد البايتات المرسلة إلى المستخدم (`"

• `bytes-out-nice` — 11352"`)عرض مقروء لعدد البايتات المرسلة إلى المستخدم (`"

• `packets-in` — 251"`)عدد الحزم المستلمة من المستخدم (`"

• `packets-out` — 211"`)عدد الحزم المرسلة إلى المستخدم (`"

• `remain-bytes-in` — البايتات المتبقية حتى بلوغ `limit-bytes-in` (`"337465"` إن لم يوجد حد `"---"` أو)

• `remain-bytes-out` — البايتات المتبقية حتى بلوغ `limit-bytes-out` (`"124455"` إن لم يوجد حد `"---"` أو)

متغيرات متنوعة (Miscellaneous variables):

• `session-id` — قيمة معامل `'session-id'` في آخر طلب

• `var` — قيمة معامل `'var'` في آخر طلب

• `error` — رسالة الخطأ عند الفشل (`"invalid username or password"`)

• `error-orig` — رسالة الخطأ الأصلية (بدون الترجمات من `errors.txt`) عند الفشل (`"invalid username or password"`)

• `chap-id` — قيمة chap ID (`"\371"`)

• `chap-challenge` — قيمة chap challenge (`"\357\015\330\013\021\234\145\245\303\253\142\246\133\175\375\316"`)

• `popup` — هل مربع اختيار الـ pop-up مفعل (`"false"` أو `"true"`)

• `advert-pending` — هل هناك advertisement بانتظار العرض (`"no"` أو `"yes"`)

• `http-status` — يسمح بتعيين http status code والرسالة

• `http-header` — يسمح بتعيين http header

متغيرات مرتبطة بـ RADIUS:

• `radius <id>` — عرض الـ attribute المحدد كـ text string (عند استخدام RADIUS authentication وإلا `""` ؛)

• `radius <id> u` — عرض الـ attribute المحدد كـ unsigned integer (عند استخدام RADIUS authentication وإلا `"0"` ؛)

• `radius <id>-<vendor>` — عرض الـ attribute المحدد مع vendor ID كـ text string (عند استخدام RADIUS authentication وإلا `""` ؛)

• `radius <id>-<vendor> u` — عرض الـ attribute المحدد مع vendor ID كـ unsigned integer (عند استخدام RADIUS authentication وإلا `"0"` ؛)

العمل مع المتغيرات (Working with variables)

`$(if <var_name> == <value>)`. يمتد أثر هذه العبارات حتى `$(elif <var_name>)` أو `$(else)` أو `$(endif)`. الشكل العام: قيمة `<var_name>` سلسلة فارغة. وهذا يكافئ `$(if <var_name> != "")`. ويمكن أيضا المقارنة بالمساواة: `$(<value>` يمكن استخدام عبارات `$(if <var_name>)` في هذه الصفحات. يدرج المحتوى التالي إذا لم تكن

```
some content, which will always be displayed
$(if username == john)
Hey, your username is john
$(elif username == dizzy)
Hello, Dizzy! How are you? Your administrator.
$(elif ip == 10.1.2.3)
You are sitting at that old computer, which is so slow...
$(elif mac == 00:01:02:03:04:05)
This is an ethernet card, which was stolen few months ago...
$(else)
I don't know who you are, so lets live in peace.
$(endif)
other content, which will always be displayed
```

سيعرض تعبير واحد فقط من هذه التعبيرات. أيها يعرض يعتمد على قيم تلك المتغيرات لكل client.

إعادة التوجيه والـ Headers المخصصة (Redirects and custom Headers)

```
$(if http-status == 302)Hotspot login required$(endif)
$(if http-header == "Location")$(link-redirect)$(endif)
```

لمتغير `"http-header"`: رغم استخدام `if`، فهو يعين المتغير إلى `"Location"` متبوعا بالـ URL من متغير `"link-redirect"`. أعلاه يبدو كأنه يستخدم تعبير الشرط `if`، فهو في الواقع *يعين* قيمة `"http-status"` إلى `"302"` وليس يختبرها. والأمر نفسه ملاحظة: رغم أن المثال

مثلا، إذا قيم `$(link-redirect)` إلى `"http://192.168.88.1/login"`، فستتغير استجابة HTTP المعادة للـ client إلى:

```
HTTP/1.0 302 Hotspot login required
<regular HTTP headers>
Location: http://192.168.88.1/login
```

صيغة `http-status`:

```
$(if http-status == XYZ)HTTP_STATUS_MESSAGE$(endif)
```

• `XYZ` — رمز الحالة الذي تريد إرجاعه. يجب أن يكون 3 أرقام عشرية، والأول يجب ألا يكون 0

• `HTTP_STATUS_MESSAGE` — أي نص تريد إرجاعه للـ client في رد HTTP بعد رمز الحالة

في أي استجابة HTTP سيكون ذلك في السطر الأول كالتالي:

```
HTTP/1.0 XYZ HTTP_STATUS_MESSAGE
```

صيغة `http-header`:

```
$(if http-header == HTTP_HEADER_NAME)HTTP_HEADER_VALUE$(endif)
```

• `HTTP_HEADER_NAME` — اسم الـ HTTP header المرسل في الاستجابة

• `HTTP_HEADER_VALUE` — قيمة الـ HTTP header ذي الاسم `HTTP_HEADER_NAME` المرسلة في الاستجابة

ستظهر استجابة HTTP كالتالي:

```
HTTP_HEADER_NAME: HTTP_HEADER_VALUE
```

تعالج جميع المتغيرات والتعابير الشرطية داخل `HTTP_HEADER_VALUE` و `HTTP_STATUS_MESSAGE` بالطريقة المعتادة.

الأخيرة فقط (وتهمل السابقة). هذا يسمح بتجاوز HTTP headers العادية (مثل `Content-Type` و `Cache-Control`).
إذا أضيفت عدة headers بنفس الاسم، تستخدم

تخصيص رسائل الخطأ (Customizing Error Messages)

لغتك. للقيام بذلك حرر `errors.txt`. ويمكنك أيضا استخدام المتغيرات داخل الرسائل. جميع التعليمات موجودة داخل ذلك الملف.
تخزن جميع رسائل الخطأ في الملف `errors.txt` داخل مجلد HotSpot servlet المعني. يمكنك تغييرها وترجمتها إلى

إصدارات متعددة لصفحات HotSpot (Multiple Versions of HotSpot Pages)

اختيارها (لاختيار اللغة) أو اختيارها تلقائيا عبر JavaScript (لاختيار نسخة PDA مقابل النسخة العادية من صفحات HTML).
يدعم وجود مجموعات صفحات HotSpot متعددة لنفس HotSpot server. يمكن للمستخدم

لعرض النسخة اللاتفية: Latviski. والنسخة اللاتفية تحتوي رابطا إلى النسخة الإنجليزية: English.
الملف المقابل من المجلد الرئيسي. عندئذ يحتوي `login.html` الرئيسي على رابط إلى `"$(link-orig-esc)/lv/login?dst="`
`radvert.html` و `errors.txt` مترجمة إلى اللاتفية. إذا لم توجد صفحة HTML المطلوبة في المجلد الفرعي المطلوب، يستخدم
إلى اللاتفية، يمكن إنشاء المجلد الفرعي `"lv"` وبداخله `login.html` و `logout.html` و `status.html` و `alogin.html` و
لاستخدام هذه الميزة، أنشئ مجلدات فرعية داخل مجلد HotSpot HTML، وضع فيها ملفات HTML المختلفة فقط. مثلا للترجمة

طريقة أخرى للإشارة إلى المجلدات هي تحديد متغير `'target'`:

```
<a href="$(link-login-only)?dst=$(link-orig-esc)&target=lv">Latviski</a>
<a href="$(link-login-only)?dst=$(link-orig-esc)&target=%2F">English</a>
```

إلى الروابط عبر متغيرات `"$(link-xxx)"`، فلا حاجة لتعديلات إضافية — سيبقى كل client داخل المجلد المختار طوال الوقت.
المحلية ذلك المسار (مثلا `"http://hotspot.mt.lv/lv/status"` = `$(link-status)`). لذا إذا أشارت كل صفحات HotSpot
بعد اختيار المجلد المفضل (مثل `"lv"`)، ستتضمن كل الروابط إلى صفحات HotSpot

متفرقات (Misc)

(التي تعتمد على `md5.js` الذي يجب أن يكون محملا مسبقا) قبل إجراء Submit لنموذج login. وإلا سيفشل CHAP login.
إذا أردت استخدام طريقة HTTP-CHAP authentication، يجب تضمين الدالة `doLogin()`

تتكون من MD5-hash لسلسلة متتابعة من: `chap-id` ثم كلمة مرور المستخدم ثم `chap-challenge` (بهذا الترتيب).
كلمة المرور الناتجة التي ترسل إلى HotSpot gateway في حالة HTTP-CHAP

وهو التمثيل الصالح لـ `"123&1=456 2"` داخل URL. يمكن تطبيق هذه الحيلة على أي متغير وليس فقط `$(username)`.
من `$(user)` استخدم النسخة المهربة: `$(user-esc)`. عندئذ يتحول نفس الـ username إلى `"123%26456%2+1D3"`
بشكل صحيح. مثلا في login page، لن يعمل الرابط كما يفترض إذا كان username هو `"123&1=456 2"`. في هذه الحالة بدلا
إذا أردت استخدام المتغيرات مباشرة داخل رابط، فيجب تهريبها (escape)

يكون `"on"` أو `"true"` لحذف cookie المستخدم عند logout (حتى لا يسجل دخوله تلقائيا عند فتح المتصفح في المرة التالية). يوجد معامل منطقي `"erase-cookie"` لصفحة logout، ويمكن أن

أمثلة (Examples)

بمعرفة أساسية بلغة HTML والأمثلة أدناه يفترض أن يسهل تطبيق الأفكار السابقة.

• لتوفير قيمة جاهزة كـ username، في `login.html` غير:

```
<type="text" value="$(username)>
```

إلى هذا السطر:

```
<input type="hidden" name="username" value="hsuser">
```

(حيث `hsuser` هو الـ username الذي توفره)

• لتوفير قيمة جاهزة كـ password، في `login.html` غير:

```
<input type="password">
```

إلى هذا السطر:

```
<input type="hidden" name="password" value="hspass">
```

(حيث `hspass` هو الـ password الذي توفره)

• لإرسال MAC address للـ client إلى خادم تسجيل بالشكل:

`https://www.example.com/register.html?mac=XX:XX:XX:XX:XX:XX`

غير رابط زر Login في `login.html` إلى:

```
https://www.example.com/register.html?mac=$(mac)
```

(صحح الرابط ليشير إلى خادمك)

• لإظهار banner بعد login، في `alogin.html` بعد

أضف السطر التالي `$(if popup == 'true')`:

```
open('http://www.example.com/your-banner-page.html', 'my-banner-name','');
```

(صحح الرابط ليشير إلى الصفحة التي تريد عرضها)

• لاختيار صفحة مختلفة بعد login، في `login.html` غير:

```
<input type="hidden" name="dst" value="$(link-orig)">
```

إلى هذا السطر:

```
<input type="hidden" name="dst" value="http://www.example.com">
```

(صحح الرابط ليشير إلى خادمك)

• لمسح cookie عند تسجيل الخروج، في الصفحة التي تحتوي رابطا إلى logout (مثلا في `status.html`) غير:

```
open('$(link-logout)', 'hotspot_logout', ...
```

إلى:

```
open('$(link-logout)?erase-cookie=on', 'hotspot_logout', ...
```

أو بدلا من ذلك أضف هذا السطر:

```
<input type="hidden" name="erase-cookie" value="on">
```

قبل هذا السطر:

<input type="submit" value="log off">

مصادقة خارجية (External authentication)

مثال آخر: جعل HotSpot يجري authentication عبر خادم بعيد (قد ينفذ مثلا charge لبطاقات ائتمان):

• اسمح بالوصول المباشر إلى الخادم الخارجي في walled-garden (سواء القائم على HTTP أو على IP)

HotSpot لتعيد التوجيه إلى خادم authentication الخارجي. يجب أن يعدل الخادم الخارجي قاعدة بيانات RADIUS حسب الحاجة
• عدل صفحة login لـ servlet

(تعيد التوجيه إلى `https://auth.example.com/login.php`؛ استبدلها بعنوان خادم authentication الخارجي الفعلي):
مثال لصفحة login كهذه توضع على HotSpot router

```
<html>
<title>...</title>
<body>
<form name="redirect" action="https://auth.example.com/login.php" method="post">
<input type="hidden" name="mac" value="$(mac)">
<input type="hidden" name="ip" value="$(ip)">
<input type="hidden" name="username" value="$(username)">
<input type="hidden" name="link-login" value="$(link-login)">
<input type="hidden" name="link-orig" value="$(link-orig)">
<input type="hidden" name="error" value="$(error)">
</form>
<script language="JavaScript">
<!--
document.redirect.submit();
//-->
</script>
</body>
</html>
```

دخول HotSpot client بإعادته إلى صفحة login الأصلية لـ HotSpot servlet مع تحديد username و password الصحيحين
• يمكن للخادم الخارجي تسجيل

استبدلها بعنوان HotSpot router الفعلي؛ وتعرض `www.mikrotik.com` بعد نجاح login، فاستبدلها بما تحتاج):
مثال لصفحة كهذه (تعيد التوجيه إلى `https://hotspot.example.com/login`؛

```
<html>
<title>Hotspot login page</title>
<body>
<form name="login" action="https://hotspot.example.com/login" method="post">
<input type="text" name="username" value="demo">
<input type="password" name="password" value="none">
<input type="hidden" name="domain" value="">
<input type="hidden" name="dst" value="http://www.mikrotik.com/">
<input type="submit" name="login" value="log in">
</form>
</body>
</html>
```

وإن لم يسمح، تعرض `flogin.html` (أو `login.html`) التي ستعيد توجيه الـ client إلى خادم authentication الخارجي.
HotSpot خادم RADIUS هل يسمح بالـ login أم لا. إن سمح، تعرض صفحة `alogin.html` (ويمكن تعديلها لفعل أي شيء).
• سيسأل

ملاحظة: كما في هذه الأمثلة، يمكن استخدام بروتوكول HTTPS وطريقة POST لتأمين الاتصالات.

اكتشاف HTTP header (HTTP header detection)

صفحات HotSpot login يمكنها الوصول إلى HTTP headers عبر `$(http-header-name)`.

وإرجاع محتوى مختلف بدل صفحة login العادية إن رغبت. يمكن استخدام ذلك لتعطيل النوافذ المنبثقة التلقائية على الهواتف مثلا.
مثلا، يمكن فحص user agent (أو المتصفح)

نسخة معينة من Firefox mobile بدل صفحة login، ضع هذه الأسطر في أعلى صفحة `rlogin.html` داخل مجلد hotspot:
مثال: لإخراج `"SUCCESS"` لمستخدمي

```
$(if user-agent == "Mozilla/5.0 (Android; Mobile; rv:40.0) Gecko/40.0 Firefox/40.0" )
<HTML><HEAD><TITLE>Success</TITLE></HEAD><BODY>Success</BODY></HTML>
$(else)
---- regular content of rlogin.html page  ----
$(endif)
```

هذا سيعطل login popup لمستخدمي Android Firefox 40.

تسجيل دخول بنقرة واحدة (One-click login)

يمكن إنشاء captive portal معدل لتسجيل دخول سريع بنقرة واحدة في السيناريوهات التي لا تتطلب user أو password.

ما تحتاج فعله:

- أنشئ مستخدما لهذا الغرض. في المثال: `"notsosecretuser"` بكلمة مرور `"notsosecretpass"`
- اربط هذا المستخدم بـ user profile يسمح بعدد محدد/غير محدود من المستخدمين النشطين المتزامنين.
- انسخ مجلد hotspot الأصلي المولد مسبقا في قائمة ملفات الموجه على المستوى الجذري.
- عدل محتويات نسخة هذا المجلد.
- ملف واحد فقط يحتاج تعديلا ليعمل هذا: `"login.html"`.

الأصلي:

```
<table width="100" style="background-color: #ffffff">
  <tr><td align="right">login</td>
     <td><input style="width: 80px" name="username" type="text" value="$(username)"/></td>
  </tr>
  <tr><td align="right">password</td>
     <td><input style="width: 80px" name="password" type="password"/></td>
  </tr>
  <tr><td> </td>
     <td><input type="submit" value="OK" /></td>
  </tr>
</table>
```

بعد التعديل:

```
<table width="100" style="background-color: #ffffff">
  <tr style="display:none;"><td align="right">login</td>
    <td><input style="width: 80px" name="username" type="text" value="notsosecretuser"/></td>
  </tr>
  <tr style="display:none;"><td align="right">password</td>
    <td><input style="width: 80px" name="password" type="password" value="notsosecretpass"/></td>
  </tr>
  <tr><td> </td>
    <td><input type="submit" value="Proceed to Internet!" /></td>
  </tr>
</table>
```

ما الذي تغير:

- حقول User و Password أصبحت مخفية.
- قيمتا حقلي User و Password تحتويان قيما جاهزة مسبقا.
- تغيرت قيمة (اسم) زر `"OK"` إلى نص أنسب.
- ارفع مجلد hotspot الجديد هذا إلى الموجه، يفضل باسم مختلف.
- غير الإعدادات في hotspot server profile لاستخدام مجلد html الجديد.

```
/ip hotspot profile set (profile number or name) html-directory-override=(dir path/name)
```

ملخص (Summary)

`ip` نفسها (مثل hosts والمستخدمين النشطين)، تضاف بعض القواعد الإضافية في جداول firewall عند تفعيل خدمة HotSpot.
إلى جانب العناصر الديناميكية الواضحة في القائمة الفرعية `hotspot/`

NAT

من أمر `ip firewall nat print dynamic/` قد ترى شيئا مثل التالي (التعليقات بعد كل قاعدة):

```
0 D chain=dstnat action=jump jump-target=hotspot hotspot=from-client
```

وضع كل مهام HotSpot المتعلقة بالحزم القادمة من جميع HotSpot clients في chain منفصلة.

```
1 I chain=hotspot action=jump jump-target=pre-hotspot
```

تحت سيطرة المسؤول بالكامل ولا تحتوي قواعد يضعها النظام؛ لذا تظهر قاعدة jump غير صالحة (لأن الـ chain فارغة افتراضيا). أي إجراءات يجب تنفيذها قبل تطبيق قواعد HotSpot توضع في `chain `pre-hotspot`. هذه الـ chain

```
2 D chain=hotspot action=redirect to-ports=64872 dst-port=53 protocol=udp
3 D chain=hotspot action=redirect to-ports=64872 dst-port=53 protocol=tcp
```

HotSpot. إذا أردت أن يستمع HotSpot server إلى منفذ آخر، أضف قواعد هنا بنفس الأسلوب مع تغيير الخاصية `dst-port`. إعادة توجيه كل طلبات DNS إلى خدمة HotSpot. المنفذ 64872 يوفر خدمة DNS لجميع مستخدمي

```
4 D chain=hotspot action=redirect to-ports=64873 hotspot=local-dst dst-port=80
  protocol=tcp
```

إعادة توجيه كل طلبات HTTP login إلى HTTP login servlet. المنفذ 64873 هو منفذ HotSpot HTTP servlet.

```
5 D chain=hotspot action=redirect to-ports=64875 hotspot=local-dst dst-port=443
  protocol=tcp
```

إعادة توجيه كل طلبات HTTPS login إلى HTTPS login servlet. المنفذ 64875 هو منفذ HotSpot HTTPS servlet.

```
6 D chain=hotspot action=jump jump-target=hs-unauth hotspot=!auth protocol=tcp
```

كل الحزم الأخرى باستثناء طلبات DNS و login من clients غير مصرح لهم يجب أن تمر عبر `chain `hs-unauth`.

```
7 D chain=hotspot action=jump jump-target=hs-auth hotspot=auth protocol=tcp
```

وحزم clients المصرح لهم — عبر `chain `hs-auth`.

```
8 D ;;; www.mikrotik.com
  chain=hs-unauth action=return dst-address=66.228.113.26 dst-port=80 protocol=tcp
```

(أي كل ما لم يضبط فيه protocol أو ضبط على TCP). هنا نستثني `www.mikrotik.com` من إعادة التوجيه إلى login page. ما يوضع في `chain `hs-unauth` هو كل ما يؤثر على بروتوكول TCP في القائمة الفرعية `ip hotspot walled-garden ip/` أول

```
9 D chain=hs-unauth action=redirect to-ports=64874 dst-port=80 protocol=tcp
```

`ip hotspot` لطلب HTTP، يمرر إلى الوجهة. وإلا يعاد توجيه الطلب تلقائيا إلى HotSpot login servlet (المنفذ 64873). إلى خادم Walled Garden proxy الذي يستمع على المنفذ 64874. إذا وجد إدخال `"allow"` في قائمة `walled-garden/` كل طلبات HTTP الأخرى تعاد توجيهها

```
10 D chain=hs-unauth action=redirect to-ports=64874 dst-port=3128 protocol=tcp
11 D chain=hs-unauth action=redirect to-ports=64874 dst-port=8080 protocol=tcp
```

المستخدم (64874) هو نفسه المستخدم لطلبات HTTP في القاعدة #9 (لذا تعالج طلبات HTTP و HTTP proxy بنفس الكود). تلقائيا علامة HTTP hotspot mark على تلك الحزم لتجاوز مشكلة الـ proxy غير المعروف، كما سنرى لاحقا. لاحظ أن المنفذ نظام HotSpot. تسمى هذه الميزة `"Universal Proxy"`. إذا اكتشف أن الـ client يستخدم proxy server معينا، سيضع النظام غير معروفة (يمكنك إضافة قواعد أخرى هنا لمنافذ إضافية). أي لتمكين العملاء ذوي إعدادات proxy غير معروفة من العمل مع أن هذه المنافذ فقط قد تستخدم لطلبات HTTP proxy. هذان الإدخالان يستخدمان "التقاط" طلبات الـ client إلى proxies يفترض HotSpot افتراضيا

```
12 D chain=hs-unauth action=redirect to-ports=64875 dst-port=443 protocol=tcp
```

.يستمع على المنفذ 64875 HTTPS proxy

```
13 I chain=hs-unauth action=jump jump-target=hs-smtp dst-port=25 protocol=tcp
```

ضبطوه على أجهزتهم [والذي قد يكون غير متاح خارج شبكة أصلهم]. الـ chain فارغة افتراضيا؛ لذا تظهر قاعدة jump غير صالحة. SMTP غير معروفة من إرسال بريدهم عبر SMTP server لمزود الخدمة (خاصتك) بدلا من الذهاب إلى SMTP server الذي

في إعدادات HotSpot. إن وجدت، توضع قاعدة redirect في chain `hs-smtp`. الهدف تمكين المستخدمين ذوي إعدادات يمكن أيضا تعريف إعادة توجيه لبروتوكول SMTP

```
14 D chain=hs-auth action=redirect to-ports=64874 hotspot=http protocol=tcp
```

للمستخدم، وكذلك على أي طلبات HTTP من مستخدمين ملفهم الشخصي مضبوط لعمل transparent proxy لطلباتهم. المضبوط على أجهزتهم [والذي قد يكون غير متاح خارج شبكة أصلهم]. تطبق هذه العلامة أيضا عندما يحين عرض advertisement proxy غير معروفة. الهدف أن يستخدم المستخدمون الذين لديهم إعدادات proxy معينة بوابة HotSpot بدلا من خادم الـ proxy HTTP proxy إلى الخوادم التي يكتشفها HotSpot HTTP proxy (المستمع على المنفذ 64874) كطلبات HTTP proxy لخوادم إلى transparent proxying (تقنية "Universal Proxy"` وميزة advertisement). توضع علامة HTTP هذه تلقائيا على طلبات توفير خدمة HTTP proxy للمستخدمين المصرح لهم. قد تحتاج طلبات المستخدمين المصادق عليهم

```
15 I chain=hs-auth action=jump jump-target=hs-smtp dst-port=25 protocol=tcp
```

توفير SMTP proxy للمستخدمين المصرح لهم (نفس فكرة القاعدة #13).

تصفية الحزم (Packet Filtering)

من أمر `ip firewall filter print dynamic/` قد ترى شيئا مثل التالي (التعليقات بعد كل قاعدة):

```
0 D chain=forward action=jump jump-target=hs-unauth hotspot=from-client,!auth
```

الموجه من client غير مصرح له ترسل إلى chain `hs-unauth`. وتنفذ `hs-unauth` مرشح Walled Garden القائم على IP. أي حزمة تعبر

```
1 D chain=forward action=jump jump-target=hs-unauth-to hotspot=to-client,!auth
```

الموجه يعاد توجيهه إلى chain أخرى اسمها `hs-unauth-to`. يجب أن ترفض هذه الـ chain الطلبات غير المصرح بها نحو العملاء. كل ما يصل إلى العملاء عبر

```
2 D chain=input action=jump jump-target=hs-input hotspot=from-client
```

كل ما يأتي من العملاء إلى الموجه نفسه يصل إلى chain أخرى اسمها `hs-input`.

```
3 I chain=hs-input action=jump jump-target=pre-hs-input
```

إلى chain خاضعة لإدارة المسؤول اسمها `pre-hs-input`، وهي فارغة افتراضيا؛ لذا تكون قاعدة jump في حالة غير صالحة. قبل المتابعة مع القواعد الديناميكية [المعرفة مسبقا]، تصل الحزمة

```
4 D chain=hs-input action=accept dst-port=64872 protocol=udp
5 D chain=hs-input action=accept dst-port=64872-64875 protocol=tcp
```

السماح للـ client بالوصول إلى خدمات authentication و proxy المحلية (كما وصف سابقا).

```
6 D chain=hs-input action=jump jump-target=hs-unauth hotspot=!auth
```

كل حركة المرور الأخرى من clients غير مصرح لهم إلى الموجه نفسه تعامل بنفس طريقة الحركة العابرة للموجهات.

```
7 D chain=hs-unauth action=return protocol=icmp
8 D ;;; www.mikrotik.com
  chain=hs-unauth action=return dst-address=66.228.113.26 dst-port=80 protocol=tcp
```

ضبطته في قائمة `ip hotspot walled-garden ip/`. لذلك رغم أنك رأيت إدخالا واحدا فقط في جدول NAT، توجد هنا قاعدتان. حيث أضيفت فقط إدخالات Walled Garden المتعلقة ببروتوكول TCP، في مرشح الحزم تضاف إلى chain `hs-unauth` كل ما بخلاف جدول NAT

```
9 D chain=hs-unauth action=reject reject-with=tcp-reset protocol=tcp
10 D chain=hs-unauth action=reject reject-with=icmp-net-prohibited
```

كل ما لم يدرج في القائمة البيضاء عبر Walled Garden سيرفض. لاحظ استخدام TCP Reset لرفض اتصالات TCP.

```
11 D chain=hs-unauth-to action=return protocol=icmp
12 D ;;; www.mikrotik.com
  chain=hs-unauth-to action=return src-address=66.228.113.26 src-port=80 protocol=tcp
```

نفس الإجراء كما في القاعدتين #7 و #8 ينفذ أيضا للحزم المتجهة إلى العملاء (chain `hs-unauth-to`).

```
13 D chain=hs-unauth-to action=reject reject-with=icmp-host-prohibited
```

رفض كل الحزم المتجهة إلى العملاء برسالة ICMP reject.